Task-3:- Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to outline the functional and non-functional requirements for the automation of a 2-tier application deployment and database backup system. This document serves as a guide for DevOps engineers and stakeholders to ensure the reliability and scalability of the Node.js and MySQL infrastructure.

1.2 Scope

- **Included:** Automation scripts for code deployment via GitHub, process management using PM2, automated MySQL backups, S3 integration, and Cron job scheduling.
- **Excluded:** Frontend development, manual OS patching, and network hardware configuration.

1.3 Definitions, Acronyms, and Abbreviations

- **EC2:** Elastic Compute Cloud (AWS Virtual Servers).
- **S3:** Simple Storage Service (AWS Cloud Storage).
- **PM2:** Production Process Manager for Node.js.
- **Crontab:** A configuration file that specifies shell commands to run periodically on a given schedule.
- **2-Tier:** An architecture consisting of a Client/Server (App) tier and a Data tier.

1.4 References

- Node.js Express MySQL GitHub Repository: [bezkoder/nodejs-express-mysql]
- AWS CLI Command Reference: [S3 sync/cp documentation]

1.5 Overview

This document covers the project objectives, specific functional automation tasks (deployment and backup), security constraints on AWS, and a risk analysis of the deployment pipeline.

---

2. Project Overview

2.1 Project Objectives

- Minimize manual intervention during application updates.
- Establish a "Point-in-Time" recovery strategy for the database.
- Ensure application high availability on AWS EC2.

2.2 Business Goals

- **Data Integrity:** Prevent data loss by maintaining hourly off-site backups.
- **Operational Efficiency:** Reduce deployment downtime through automated shell scripts.

2.3 Background

The 2-tier application is a Restful CRUD API. To move from a manual "developer-managed" state to a "DevOps-managed" state, the infrastructure requires automated pipelines to handle code changes and data persistence.

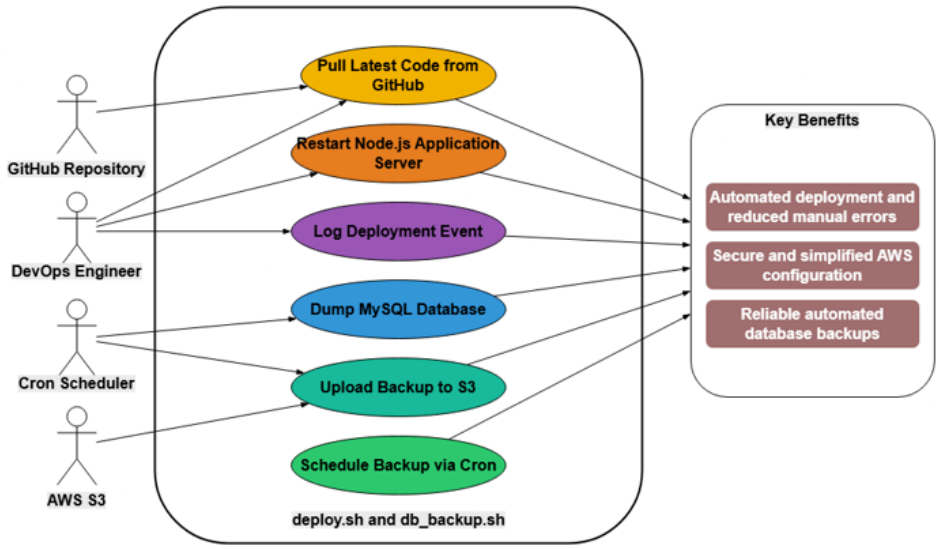
2.4 Constraints

- **Technology:** Must use Linux Shell scripts and AWS CLI.
- **Architecture:** Strictly 2-tier (separate App and DB instances).
- **Time:** Backups must trigger exactly every 60 minutes.

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Case 1: Automated Deployment (`deploy.sh`)

- **Description:** Pulls the latest code and restarts the server.
- **Actors:** DevOps Engineer / System Trigger.
- **Preconditions:** Git is configured; PM2 is running the initial process.
- **Postconditions:** App is running the latest version; logs are updated.
- **Flow of Events:**

a. Enter the application directory.
b. Execute `git pull`.
c. Run `npm install` for dependency updates.
d. Restart process via `pm2 restart`.
e. Append status to `deployment.log`.

3.3 Use Case 2: Database Backup (`db_backup.sh`)

- **Description:** Backs up MySQL and uploads to S3.
- **Actors:** Cron Daemon.
- **Preconditions:** AWS CLI configured with S3 permissions.
- **Postconditions:** A `.sql` file exists in the S3 bucket; local temp file removed.
- **Flow of Events:**
  a. Run `mysqldump` with a timestamped filename.
  b. Transfer file to S3 using `aws s3 cp`.
  c. Verify upload success.
  d. Delete local copy to save disk space.

---

4. Non-Functional Requirements

4.1 Performance Requirements

- Deployment script must complete execution in under 2 minutes.
- Backup process should not cause significant CPU spikes on the DB instance.

4.2 Security Requirements

- Database credentials must not be hardcoded in plain text (use environment variables or IAM roles).
- The S3 bucket must have restricted public access.

4.3 Usability Requirements

- Scripts must be idempotent (running them multiple times should not break the system).
- Logs must be human-readable for troubleshooting.

4.4 Reliability Requirements

- **Uptime:** Application must restart automatically if it crashes (handled by PM2).
- **Recovery:** Backup files must be valid and restorable.

---

5. Assumptions and Dependencies

5.1 Assumptions

- The EC2 instances have internet access to reach GitHub and S3.
- The user has sudo privileges on both EC2 instances.

5.2 Dependencies

- **GitHub:** Availability of the repository to pull code.
- **AWS IAM:** The instance requires an IAM Role with `S3PutObject` permissions.

---

6. Risk Analysis

6.1 Risk Identification

1. **Disk Full:** Backups failing because the DB server runs out of space.
2. **Network Partition:** App server cannot connect to DB server after a restart.

6.2 Risk Mitigation

1. **Mitigation:** Add a command in `db_backup.sh` to delete local files immediately after S3 upload.
2. **Mitigation:** Configure Security Groups to allow persistent traffic on Port 3306.

---

7. Glossary

- **IAM:** Identity and Access Management.
- **EBS:** Elastic Block Store (EC2 Hard Drive).
- **CRUD:** Create, Read, Update, Delete.

---

8. Appendices

- **Sample Crontab Entry:** `0 * * * * /home/ubuntu/db_backup.sh`
- **Proof of Success:** See uploaded screenshots for S3 bucket contents and PM2 status.

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____